hardware platform. It can also choose the proper execution path or run-time libraries to leverage specific processor features or do anything associated with the processor information.

Standard and extended functions

The CPUID instruction supports two sets of functions: standard and extended functions.

For standard functions -- the highest acceptable value (or range) for the EAX register input and CPUID functions that return the basic processor information—the program should input 0000_0000h to the EAX register.

```
mov eax, 00h
cpuid
```

For extended functions—the highest acceptable value (or range) for the EAX register input and CPUID functions that return the extended processor information—the program should input 8000_0000h to the EAX register.

```
mov eax, 80000000h
cpuid
```

After the execution of the above CPUID instructions, the result will be present in the EAX register.

(For more detailed information about standard and extended functions, readers can refer to the Intel Architecture Software Developer's Manual, or the AMD64 Architecture Programmer's Manual.)

CPUID with the GCC inline assembly function

C professionals/hobbyists can exploit CPUID instruction with the GCC inline assembly function, that is, asm.

```
__asm__ ("cpuid"
        :"=a" (result[EAX]),
        "=b" (result[EBX]),
        "=c" (result[ECX]),
        "=d" (result[EDX])
        :"a" (input_eax)
        );
```

The GNU C compiler uses the AT&T syntax for assembly coding. To learn more about the inline assembly feature provided by GCC, you may refer to *www.ibiblio.org/gferg/ldp/GCC-Inline-Assembly-HOWTO.html*.

Now let's try out something more significant with CPUID using the inline assembly function.

The CPU vendor string, however, may not be extremely valuable to most developers, but it is easy to demonstrate how CPUID instruction fetches information with just a basic example.

With an input value of 0000_0000h in EAX, in addition to returning the largest standard function number in the EAX register (as discussed above), the CPU vendor ID string can also be verified at the same time. We can see how, with this example:

```
#include <stdio.h>
#define EAX 0
#define EBX 1
#define ECX 2
#define EDX 3

int main()
{
  unsigned int result[4], input_eax;
  input_eax = 0x00;
  __asm__ ("cpuid"
        :"=a" (result[EAX]),
        "=b" (result[EBX]),
        "=c" (result[ECX]),
        "=d" (result[EDX])
        :"a" (input_eax)
        );
  return 0;
}
```

In the above C code, after the ***__asm__*** function executes the CPUID instruction, ***result[EBX], result[ECX]*** and ***result[EDX]*** (when combined) will represent the complete vendor ID string. (Readers should easily be able to run the above example on any GCC compiler.)

On my Intel processor-based laptop, the vendor-ID string returned is "GenuineIntel" as these registers contain the following values:

- EBX → 0x756e6547 → "uneG" ('G' in BL),
- EDX → 0x49656e69 → "Ieni" ('i in DL),
- ECX → 0x6c65746e → "letn" ('n' in CL).

When the CPUID instruction is executed with 1 in EAX, the version and feature information is returned back to EAX.

For detailed information about processor identification and supported features (standard and extended functions), readers may refer to the Intel Architecture Software Developer's Manual or the AMD64 Architecture Programmer's Manual. END

References

- AMD CPUID Specification, Rev. 2.28: *www.amd.com/us-en/assets/content_type/white_papers_and_tech_docs/25481.pdf*
- Intel Processor Identification and the CPUID Instruction: *www.intel.com/Assets/PDF/appnote/241618.pdf*
- Intel 64 and IA-32 Architecture Software Developer's Manual Volume 2A: *Instruction Set Reference, A-M: http://www.intel.com/assets/pdf/manual/253666.pdf*

By: Vivek Kumar

The author is a system engineer at IBM India, and works for IBM-ISS (Internet Security System). He is based in Pune and can be reached at mail_DOT_vivekkumar_AT_rediffmail_DOT_com"